Bayesian-Optimized Variational Quantum Circuits

Teaching a 2-qubit circuit to classify data, without gradients

Jialun Li Juan Vera

PHYS 126

October 14, 2026

The question

Can a **tiny quantum circuit** learn to classify data as well as a **small neural network**...

...if we train it without gradients?

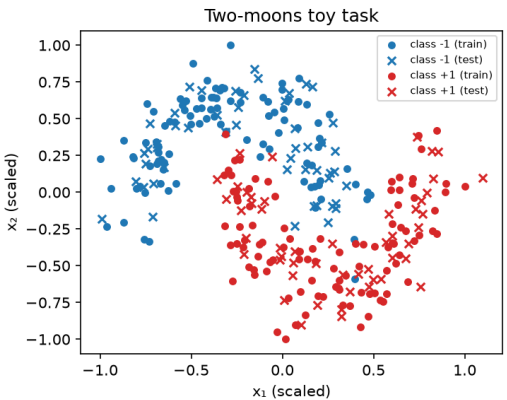
Why "without gradients"?

- Neural nets are trained by following the slope of the error.
- On a real quantum computer, that slope is *expensive and noisy* to measure.
- So people want optimizers that only need the score, not the slope.

What we did

- Built a 2-qubit circuit in Qiskit.
- Trained it with **Bayesian optimization**.
- Raced it against classical neural networks on the same task.

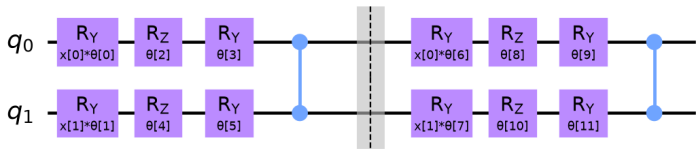
The task: two moons



- 300 points in two interleaved crescents, red and blue.
- Goal: given a new point, say which colour it is.
- 200 points to learn from, 100 held back to test on.

No straight line separates them. A straight-line classifier scores 84 %. Everything above that needs a *curved* boundary.

What a quantum classifier is



two of the four layers shown

1. Rotate each qubit by an angle that depends on the data point (x_1, x_2).
2. Rotate again by **trainable** angles θ (the "knobs").
3. Entangle the two qubits. Repeat four times.
4. Measure qubit 0.

Output: a number $f \in [-1, 1]$.
$f > 0 \Rightarrow$ red, $f < 0 \Rightarrow$ blue.

24 knobs in total. **Training** = turning the knobs until the sign is right for as many points as possible.

Why we can't just use gradients

Neural network

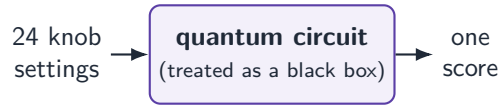
- Backpropagation: one backward pass gives the slope for *every* weight.
- Cost $\approx$ one forward pass.

Quantum circuit

- You can't look inside the state without destroying it, so there is no backward pass.
- The slope for one knob costs two extra full runs:

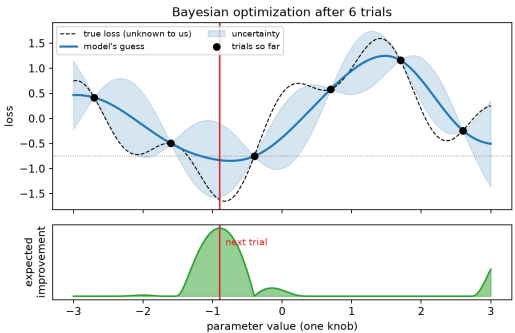
$$\frac{\partial f}{\partial \theta} = \tfrac{1}{2}\left[f\left(\theta + \tfrac{\pi}{2}\right) - f\left(\theta - \tfrac{\pi}{2}\right)\right]$$

- 24 knobs $\Rightarrow$ **48 extra runs** per data point



So we treat it as a black box: knobs in, score out. We need an optimizer that works from scores alone.

Bayesian optimization in plain words



You can only afford a few hundred trials. After each one:

1. Fit a smooth guess through every result so far, *with error bars*.
2. Try next wherever the guess is low **or** the error bars are wide.

That trade-off has a name, *Expected Improvement*:

$$\mathrm{EI} = \underbrace{(\mathcal{L}^* - \mu)\,\Phi(z)}_{\text{probably good}} + \underbrace{\sigma\,\varphi(z)}_{\text{haven't looked}}$$

Every trial is chosen by the model, not by a fixed rule.

What we compared

Quantum circuit (24 knobs), trained by

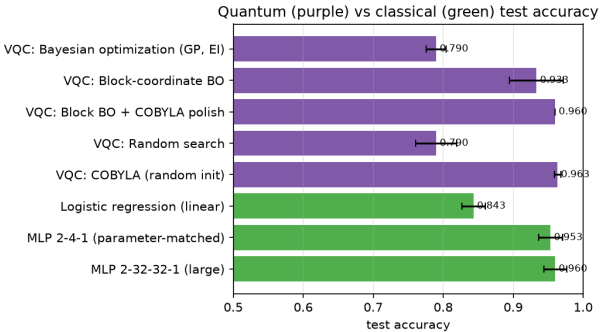
- Bayesian optimization over all 24 knobs at once
- Bayesian optimization on **6 knobs at a time**, cycling through the layers
- …plus a short local "polish" step at the end
- Random guessing (control)
- COBYLA, a standard local optimizer (control)

Classical models, trained with gradients

- A straight line — 3 weights
- A tiny neural net — 17 weights
- A big neural net — 1185 weights

Every quantum optimizer gets the same budget of circuit runs. Everything is repeated for 3 random seeds and averaged.

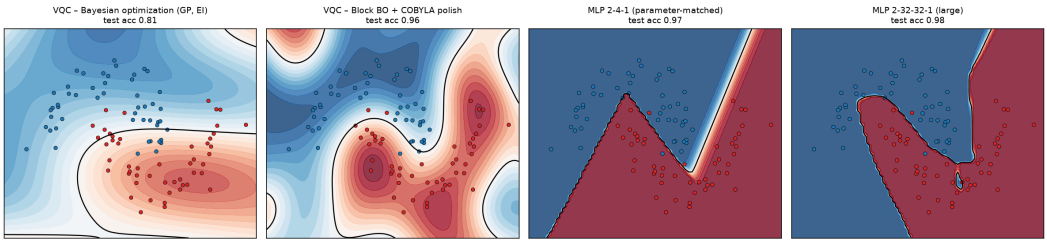
Results



1. **The quantum circuit reaches 96 %**, the same as the neural networks.
2. BO over all 24 knobs at once was **no better than random guessing** (79 %).
3. BO on 6 knobs at a time fixed it (93 %); the polish step made it reliable (96 %).

Still 95 % when read out with only 256 measurement shots per point.

What the models learned



Left to right: the boundary goes from a blob (BO on all 24 knobs) to a curve that hugs the moons (BO on 6 at a time + polish).

The quantum boundary is **smooth and wavy**; the neural net's is made of **straight segments**. Same problem, different shapes.

Honest conclusion, and what's next

- A 2-qubit circuit with 24 knobs, trained with **no gradients at all**, matched classical neural networks with 17 and 1185 weights.
- Bayesian optimization only works once you keep it to a handful of knobs at a time.
- **The catch:** on our noise-free simulator, the simpler local optimizer COBYLA did just as well.

Next experiment: add measurement noise to training. That is where a local optimizer should fall apart and Bayesian optimization should earn its keep.

Not a quantum advantage, and none expected at 2 qubits. The point was the *training method*.

Code, results and a live terminal demo: `bo-vqc` repository